

RISC-V SECURE IoT GATEWAY SoC

ARCHITECTURE DOCUMENT

RISC-V Based SoC with I2C, SPI, DMA and AES-128 Hardware Security

1. System Architecture

The proposed Secure IoT Gateway is organized around a RISC-V processor that controls sensor acquisition, data movement, encryption, and host communication. Instruction Memory and Data Memory provide the processor with program and data storage. The processor accesses the memories and memory-mapped peripherals through an AXI4 system interconnect. The four application-specific IPs are an I2C Controller, SPI Controller, DMA Controller, and AES-128 Accelerator.

The architecture separates control-plane and data-plane responsibilities. RISC-V performs initialization, configuration, sequencing, status handling, and lightweight software processing. I2C/SPI handle sensor communication, DMA handles block data movement, and AES-128 performs cryptographic acceleration. UART provides host/debug output, while Timer and GPIO support timing and status.

2. Block Diagram

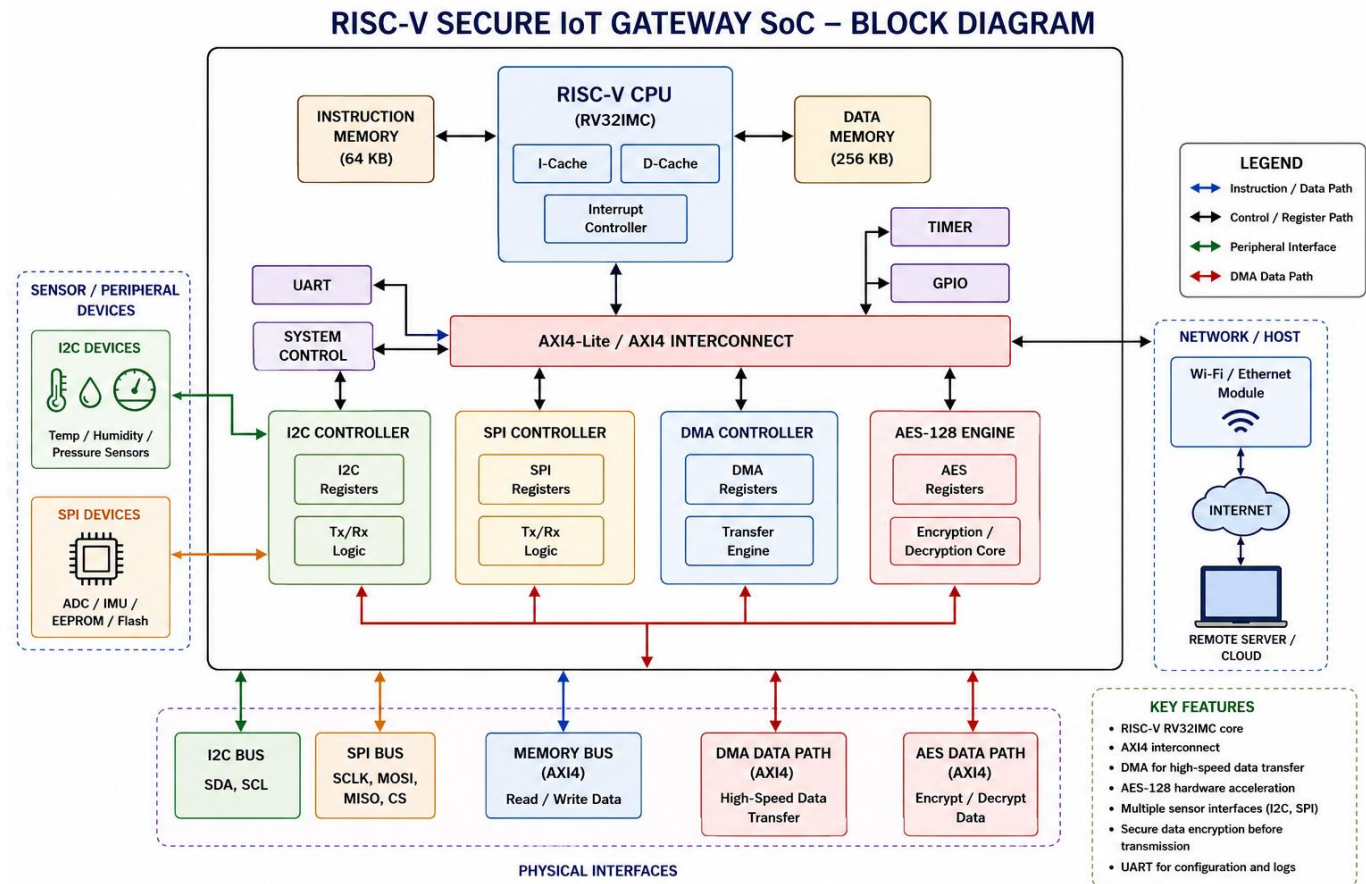


Figure 1. Proposed top-level architecture and application data path.

3. Memory Map

The following 32-bit memory map is a proposed project architecture. The exact address ranges should be frozen after the selected RISC-V core and AXI implementation are finalized.

Address Range	Size	Target	Purpose
0x0000_0000– 0x0000_FFFF	64 KB	Instruction Memory	RISC-V firmware/program storage
0x1000_0000– 0x1000_FFFF	64 KB	Data Memory	Sensor data, DMA buffers, AES data and software variables
0x2000_0000– 0x2000_00FF	256 B	UART	TX/RX data and status
0x2000_0100– 0x2000_01FF	256 B	Timer	Counter, control and status
0x2000_0200– 0x2000_02FF	256 B	GPIO	Data, direction and status
0x4000_0000– 0x4000_00FF	256 B	I2C Controller	I2C control, status and data
0x4000_0100– 0x4000_01FF	256 B	SPI Controller	SPI control, status and data
0x4000_0200– 0x4000_02FF	256 B	DMA Controller	Source/destination/length and transfer status
0x4000_0300– 0x4000_03FF	256 B	AES-128 Accelerator	Key, plaintext, ciphertext and control/status

4. Interface Specifications

4.1 RISC-V / AXI4 System Interface

- RISC-V acts as the main control master for normal instruction and data transactions.
- The system interconnect decodes the address and routes each transaction to Instruction Memory, Data Memory, UART, Timer, GPIO, I2C, SPI, DMA, or AES.
- The processor accesses peripheral registers through memory-mapped reads and writes.
- AXI4 read and write transactions use independent address/data channels and VALID/READY handshaking.

4.2 I2C Controller Interface

Interface	Signals / Function
System side	AXI4-Lite-style memory-mapped register interface for configuration/status
Serial side	SCL and SDA
Control	ENABLE, START, READ/WRITE, device address, clock

	divider
Data	TX_DATA and RX_DATA
Status	BUSY, DONE, ACK_ERROR
Optional	IRQ/DONE interrupt

4.3 SPI Controller Interface

Interface	Signals / Function
System side	AXI4-Lite-style memory-mapped register interface
Serial side	SCLK, MOSI, MISO, CS
Control	ENABLE, START, CPOL, CPHA, clock divider
Data	TX_DATA and RX_DATA
Status	BUSY, DONE, ERROR
Optional	Transfer-complete interrupt

4.4 DMA Controller Interface

Interface	Signals / Function
CPU side	Memory-mapped configuration registers
AXI master side	Read/write access to Data Memory and mapped transfer targets
Configuration	SRC_ADDR, DST_ADDR, LENGTH, CONTROL
Status	BUSY, DONE, ERROR
Transfer mode	Incrementing source/destination block transfer
Optional	Peripheral request/handshake and completion interrupt

4.5 AES-128 Accelerator Interface

Interface	Signals / Function
System side	Memory-mapped control/status/data registers
Key	128-bit AES key
Input	128-bit plaintext block
Output	128-bit ciphertext block
Control	START / ENABLE

Status	IDLE / BUSY / DONE / ERROR
Optional	Completion interrupt

4.6 UART / Timer / GPIO

- UART provides host-side encrypted-data output, debugging, and status reporting.
- Timer provides a programmable counter for execution and end-to-end latency measurement.
- GPIO provides general-purpose status/control outputs such as acquisition, DMA, AES, and error indicators.

5. Bus Protocol Details

AXI4 is selected as the primary system interconnect for this architecture. The bus separates read and write transactions and uses independent channels. A transfer on each channel occurs when both VALID and READY are asserted in the same clock cycle.

5.1 AXI4 Write Channels

Channel	Important Signals	Purpose
AW – Write Address	AWADDR, AWVALID, AWREADY, AWLEN, AWSIZE, AWBURST	Carries write address and burst attributes
W – Write Data	WDATA,WSTRB, WVALID, WREADY, WLAST	Carries write payload, byte strobes and final-beat indication
B – Write Response	BRESP, BVALID, BREADY	Returns completion/error response to the write transaction

5.2 AXI4 Read Channels

Channel	Important Signals	Purpose
AR – Read Address	ARADDR, ARVALID, ARREADY, ARLEN, ARSIZE, ARBURST	Carries read address and burst attributes
R – Read Data	RDATA, RRESP, RVALID, RREADY, RLAST	Returns read data and response status

5.3 VALID / READY Handshake

The sender asserts VALID when the channel information is available. The receiver asserts READY when it can accept the transfer. A channel transfer takes place on a clock edge where VALID = 1 and READY = 1. VALID must not depend combinationally on READY in a way that creates a deadlock.

5.4 Address Decoding

The interconnect examines the address of each CPU transaction and selects one target. For example, addresses in the 0x4000_0000–0x4000_00FF range select the I2C controller, while 0x4000_0300–0x4000_03FF select the AES-128 accelerator. The selected slave returns the read data or write response through the interconnect.

5.5 Example Register Write

To start DMA, the RISC-V writes SRC_ADDR, DST_ADDR, and LENGTH, then writes the START bit in CONTROL. The AXI write address identifies the DMA register, the write data updates the register, and the DMA returns a write response. DMA then performs the programmed transfer and asserts DONE when LENGTH has been completed.

5.6 Example Register Read

To check AES completion, the RISC-V reads the AES STATUS register. The AXI read address is decoded to the AES slave, the AES returns STATUS through the R channel, and the processor determines whether the accelerator is IDLE, BUSY, DONE, or in an error state.

5.7 Application Data Flow

Sensor → I2C/SPI → DMA → Data Memory → AES-128 → Data Memory → DMA/UART → Host

6. Architecture Summary

The architecture combines the RISC-V processor and standard SoC infrastructure with four application-specific IP blocks: I2C, SPI, DMA, and AES-128. The resulting system provides a complete secure IoT gateway data path in which sensor data is acquired, transferred efficiently, encrypted in hardware, and made available to a host. The proposed memory map, interfaces, and AXI4 protocol details provide the baseline specification for RTL implementation and verification.